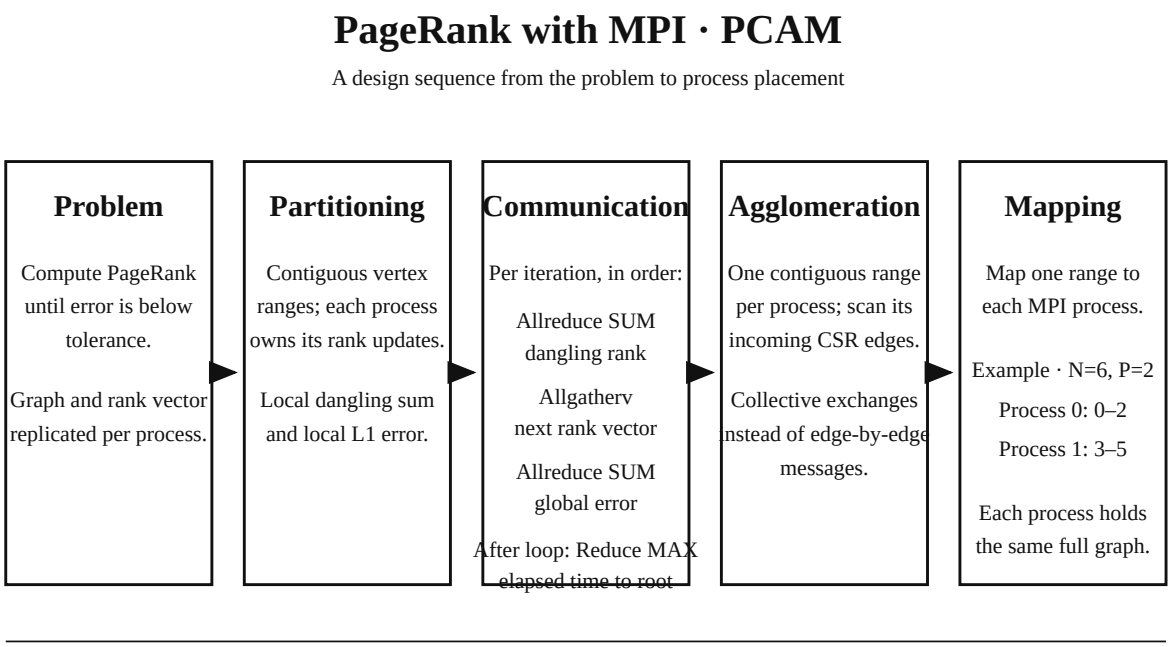
Module III · MPI PageRank

Mohammed Ashar Husain **Roll No. 2024bcs0137**

Copy-ready study guide · typed reference, not a handwritten submission

Q6 · Parallelizable blocks — Foster PCAM

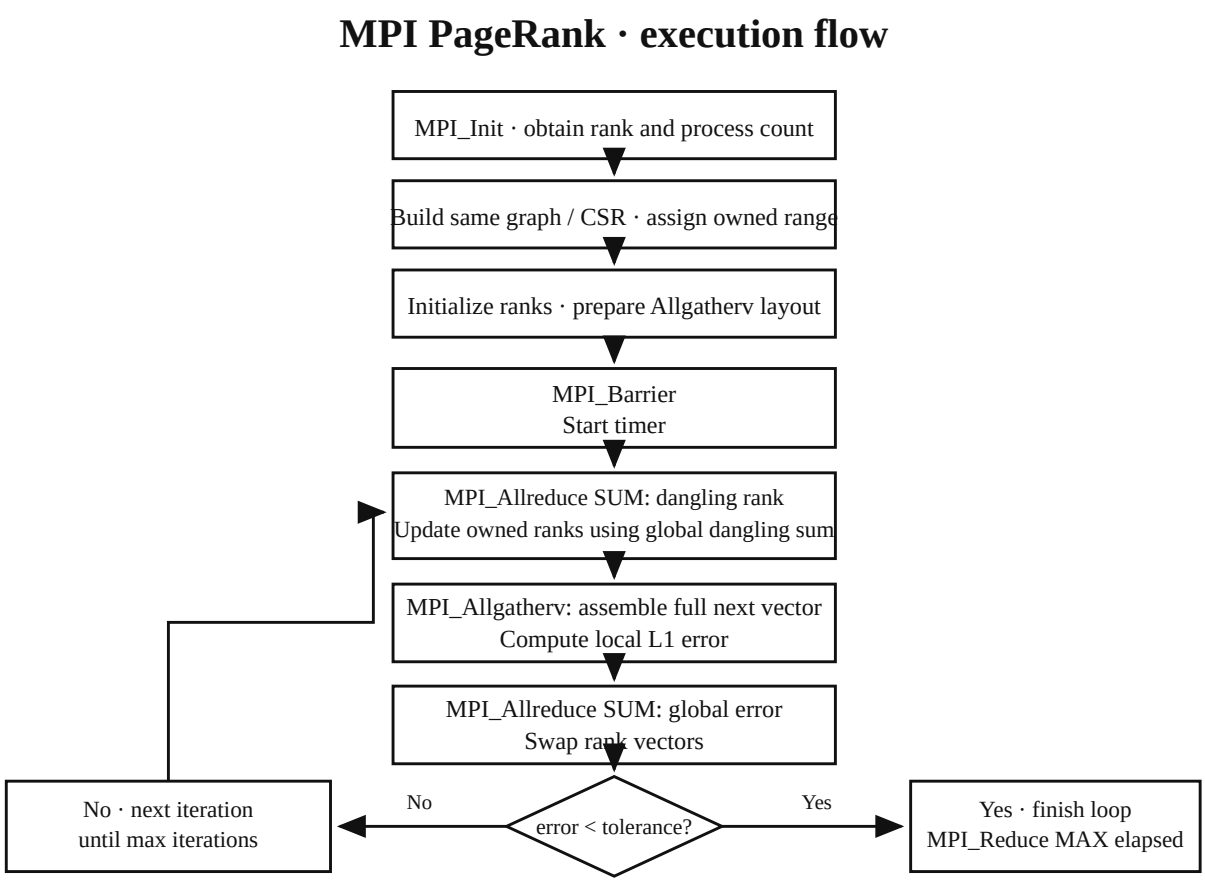
Each destination update is independent within an iteration once current ranks and total dangling rank are known. Iterations depend on one another; all processes need the same convergence decision.



PCAM sequence and MPI collectives · reference to copy by hand

Partition and mapping (N=10, P=4): all processes store the same graph and full rank vector, but update only their contiguous range. Process 0: vertices 0–1 [0,2); process 1: 2–4 [2,5); process 2: 5–6 [5,7); process 3: 7–9 [7,10). The owner scans incoming CSR edges for its destinations.

Q7 · MPI pseudocode



MPI PageRank iteration flow · reference to copy by hand

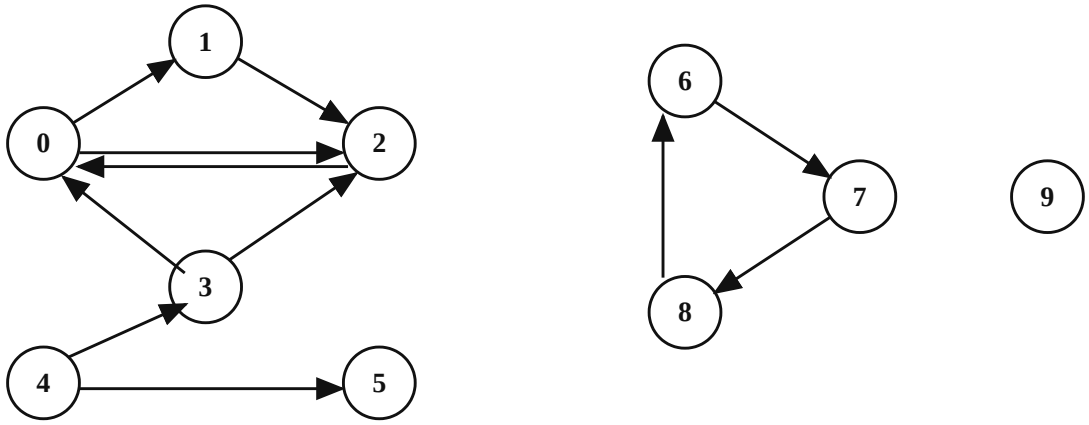
```
MPI_Init; get rank p and process count P
Build and validate the same graph on every process
Set owned range [begin,end); initialize ranks to 1/N
MPI_Barrier; start = MPI_Wtime()
for iteration = 1 .. 10,000:
    local_dangling = sum owned dangling ranks
    D = MPI_Allreduce(local_dangling, SUM)
    base = (1-d)/N + d*D/N
    update each owned destination using incoming edges
    MPI_Allgatherv(next_local, next)
    local_L1 = sum owned |next[v]-rank[v]|
    L1 = MPI_Allreduce(local_L1, SUM); swap(rank,next)
    if L1 < 1e-6: break
local_elapsed = MPI_Wtime() - start
MPI_Reduce(local_elapsed, MAX, root=0)
Rank 0 prints result; free memory; MPI_Finalize
```

Q8 · N10 demonstration — first iteration

The graph has N=10, E=11. Vertex 5 has no outgoing edge; vertex 9 is isolated. Separate clusters are not connected.

Directed graph · N = 10

Connected component · vertices 0–5 **Cycle · vertices 6–8** **Isolated**



5 · dangling (no outgoing edges)

Exact directed edge list (11 edges)

0→1, 0→2, 1→2, 2→0, 3→0, 3→2, 4→3, 4→5, 6→7, 7→8, 8→6

No other edges · 5 is dangling · 9 is isolated

Exact directed N10 graph · reference to copy by hand

Edges: 0→1, 0→2, 1→2, 2→0, 3→0, 3→2, 4→3, 4→5, 6→7, 7→8, 8→6.

With four processes, every initial rank is 0.1. Local dangling ranks by process are 0, 0, 0.1 (vertex 5), and 0.1 (vertex 9); total D=0.2. Thus base=(1−0.85)/10+0.85×0.2/10=0.032.

Examples: next[0]=0.032+0.85(0.1/1+0.1/2)=0.1595; next[2]=0.032+0.85(0.1/2+0.1/1+0.1/2)=0.2020.

The gathered next ranks, vertex order: `.1595, .0745, .2020, .0745, .0320, .0745, .1170, .1170, .1170, .0320`. Global L1 change=0.425, so continue. Actual one-iteration capped output: `10,11,1,0.000027189,max_iterations,0.42499999999999999`. It is not converged.

Q9 · MPI source, run and actual output

Source: `c/mpi_pagerank.c`; graph, N10 fixture and constants: `c/shared_graph.h`.

```
mpicc -O2 -std=c11 -Wall -Wextra -pedantic c/mpi_pagerank.c -o outputs/bin/mpi_pagerank
mpirun -np 2 outputs/bin/mpi_pagerank --case N10 --emit-ranks
```

Actual converged MPI/2 result row from `outputs/mpi_output.txt`:

`10,11,25,0.000064743,success,5.6465066086697213e-07`

Columns are N, E, iterations, seconds, status, final L1 error. A separate `ranks,...` row lists ranks in vertex order. For the hardcopy, attach the printed full source and output, labelled with the name and roll number above; those identifiers are not program output. The assignment PDF's Q9a says “OpenMP” despite its MPI heading; this answer follows the heading.

Q10 · Time and space complexity

Let N be vertices, E edges, I iterations, P processes. Serial work is O(I(N+E)); balanced MPI computation is approximately O(I(N+E)/P) per process. Each iteration adds two scalar Allreduces and an Allgatherv of N ranks to every process. Imbalance and communication can limit speedup.

Each process stores replicated CSR and full rank vectors: O(N+E) memory per process, O(P(N+E)) total. The graph is not distributed. Timing starts after a barrier, before the loop; reported elapsed time is the maximum across processes.

Q11 · Execution-time comparison

Median seconds from five timed readings per configuration. OpenMP/4 means four threads; MPI/2 and MPI/4 mean two and four processes. Copy the table by hand. Attach the actual comparison graph from the separate print packet; if drawing it, label the horizontal axis N and the vertical axis median seconds (log scale).

N (E)	Serial	OpenMP/4	MPI/2	MPI/4
10 (11)	1.587e−6	5.548e−4	6.474e−5	1.387e−4
100 (594)	8.786e−5	1.008e−3	1.163e−4	2.849e−4
1,000 (5,994)	4.410e−4	1.009e−3	2.561e−4	3.508e−4
10,000 (59,995)	1.830e−3	1.991e−3	1.255e−3	1.029e−3
100,000 (599,992)	9.252e−3	8.243e−3	6.543e−3	5.660e−3

At N=100,000, MPI/4 speedup=0.009252/0.005660=1.635×; efficiency=1.635/4=0.409 (about 40.9%). Small graphs may run slower in parallel because overhead exceeds saved computation. Results are machine-specific, not universal.